자료구조 실습

09/30

CONTENTS

오늘 실습 순서

- 코드 구조 확인 — SortedList의 data · length, 4주차 UnsortedList 복습
- 문제 1~4: 문제를 먼저 보고 직접 작성한 뒤, 같이 풀이 확인
- 1) insertItem 2) findItem(이진 탐색)
- 3) removeItem 4) updateItem
- 문제 5: 정렬 vs 비정렬 비교 실험 — 탐색과 삽입 비용을 직접 측정
- 연산별 Big-O 총정리

HOW-TO

실습 진행 방법

- practice.py를 열고 문제 1~4의 NEED_TO_SOLVE를 채움
- NEED_TO_SOLVE는 아무 동작도 하지 않는 빈 자리 -> 실행은 되지만 결과가 틀림
- 실행하면 체크포인트가 PASS / FAIL로 나옴. 전부 PASS면 문제 5 실험이 이어서 실행됨

코드 받기

```
git clone https://github.com/msforgood/data-structure-assist.git


# 지난주에 이미 받았다면
git pull origin main
```

실행

```
cd 2026-가을/w5-0930
python3 practice.py


# 정답 코드: practice_answer.py
```

규칙

```
AI 사용 금지.
파이썬 list의 append / insert / remove / index / in / sort, bisect 모듈 사용 금지
-> self.data 배열과 self.length만 직접 조작
```

REFERENCE · practice.py

이번 주에 다룰 클래스 — SortedList

- data[0] ~ data[length-1]은 항상 오름차순 -> 이 약속 덕분에 이진 탐색이 가능해짐

이미 작성되어 있는 부분

```
def __init__(self, max_size=MAX_SIZE):
    self.data = [None] * max_size   # 저장 공간 (크기 고정)
    self.length = 0                 # 지금 들어 있는 원소 개수
```

- 주어진 것 : size, isFull, isEmpty, getItem, clear, toList
- 채울 것 : insertItem, findItem, removeItem, updateItem (문제 1~4)
- 4주차 UnsortedList도 들어있음(appendItem·findItem) — 비교 기준으로만 사용

REFERENCE · practice.py

정렬 리스트에서 달라지는 것 — 4주차와 비교

- 이름은 같은 연산이라도, 정렬 순서를 지켜야 하므로 안에서 하는 일이 달라짐

4주차(비정렬) vs 이번 주(정렬)

연산	4주차 UnsortedList	이번 주 SortedList
`insertItem`	아무 자리에나 넣으면 됨 (pos 지정)	값이 들어갈 위치를 찾아서 끼워 넣어야 함
`findItem`	0번부터 하나씩 비교 (선형 탐색)	반씩 줄여가며 탐색 (이진 탐색)
`removeItem`	지운 자리를 맨 뒤 값으로 메움	지운 자리부터 끝까지 당겨야 함
`updateItem`	(4주차엔 없음)	값을 바꾸되 정렬 순서를 지켜야 함

정렬을 유지하는 대가로 탐색이 빨라짐 -> 문제 5에서 직접 측정해서 확인

REFERENCE

bisect · sort를 쓰지 않는 이유

- bisect.insort·list.sort() 한 줄이면 끝나지만, 몇 번 비교·이동했는지는 안 보임
- 이번 주에 세는 것도 '계산 횟수' -> 인덱스를 직접 다뤄야 횟수가 보임

사용 금지

```
bisect.bisect_left(data, v)
bisect.insort(data, v)
list.sort() / sorted(data)
v in lst / lst.index(v)
```

이렇게 씀

```
if item < self.data[mid]:
    last = mid - 1
while i > location:
    ...
```

- self.data 자체는 파이썬 리스트지만, 여기서는 '크기가 고정된 배열'로만 씀
- 인덱스로 읽고 쓰는 것(data[i])만 허용 -> 크기·정렬 기능은 사용하지 않음

문제 1 · 소개

insertItem( ) — 정렬 순서를 지키며 삽입

완성할 코드 (practice.py)

```
def insertItem(self, value):
    location = 0
    while NEED_TO_SOLVE:    # data[location] >= value인 첫 자리
        location += 1
    i = self.size()
    while NEED_TO_SOLVE:    # 뒤에서부터 한 칸씩 밀기
        NEED_TO_SOLVE
        i -= 1
    NEED_TO_SOLVE = value    # 삽입
    self.length += 1
```

- [1, 4, 5]에 insertItem(3)을 하면 결과는? (정확한 반복문은 practice.py 참고)

⏱️ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

문제 1 · 풀이 · insertItem( )

위치를 찾고, 뒤에서부터 밀고, 끼워 넣기

풀이 코드

```
location = 0
while (location < self.size()
       and self.data[location] < value):
    location += 1
i = self.size()
while i > location:
    self.data[i] = self.data[i - 1]
    i -= 1
self.data[location] = value
self.length += 1
```

insertItem(3) 예시 — [1, 4, 5]

```
위치 찾기 location=1 (4>=3)
밀기 [1,4,5,5] -> [1,4,4,5]
삽입 [1,3,4,5]  length=4
```

실행 결과

```
[PASS] insertItem 3,5,1,4,2 삽입: [1, 2, 3, 4, 5]
```

문제 2 · 소개

findItem( ) — 이진 탐색으로 찾기

- 정렬되어 있으므로 가운데 값과 비교해서 탐색 범위를 반씩 줄일 수 있음
- 4주차 findItem(0번부터 하나씩)과 이번 문제를 비교해볼 것

완성할 코드 (practice.py)

```
def findItem(self, item):
    first, last = NEED_TO_SOLVE, NEED_TO_SOLVE
    while NEED_TO_SOLVE:
        mid = NEED_TO_SOLVE
        NEED_TO_SOLVE    # mid와 item 비교 -> first/last 좁히거나 반환
    return -1
```

⏱️ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

문제 2 · 풀이 · findItem( )

가운데 값과 비교해 범위를 반씩 줄임

풀이 코드

```
first, last = 0, self.size() - 1
while first <= last:
    mid = (first + last) // 2
    if item < self.data[mid]:
        last = mid - 1
    elif item > self.data[mid]:
        first = mid + 1
    else:
        return mid
return -1
```

findItem(4) 예시 — [1,2,3,4,5]

```
mid=2 data[2]=3<4 -> first=3
mid=3 data[3]=4==4 -> return 3
이진 탐색 2번 비교 (선형은 4번)
범위가 절반씩 줄어듦 -> O(log N)
```

실행 결과

```
[PASS] findItem(4): 3
[PASS] findItem(10): -1
```

문제 3 · 소개

removeItem( ) — 찾아서 지우고, 당겨서 메우기

- ① findItem으로 찾기 → ② 그 위치부터 끝까지 왼쪽으로 당기기 → ③ length 감소
- 4주차처럼 맨 뒤 값으로 덮어쓰면 정렬이 깨짐 -> 이번엔 왜 반드시 당겨야 하는가?

완성할 코드 (practice.py)

```
def removeItem(self, value):
    # (findItem으로 idx 찾는 부분은 이미 작성되어 있음)
    i = idx
    while NEED_TO_SOLVE:    # idx 이후를 한 칸씩 왼쪽으로 당김
        NEED_TO_SOLVE
        i += 1
```

⏱️ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

문제 3 · 풀이 · removeItem( )

지운 자리부터 끝까지 한 칸씩 당김

풀이 코드

```
i = idx
while i < self.size() - 1:
    self.data[i] = self.data[i + 1]
    i += 1

self.data[self.size() - 1] = None
self.length -= 1
```

removeItem(3) 예시 — [1,2,3,4,5]

```
찾기    findItem(3) -> 2
당기기  data[2]<-data[3]
        data[3]<-data[4]
        [1, 2, 4, 5]  length=4
```

4주차처럼 맨 뒤 값을 그 자리로 옮기면 [1, 2, 5, 4]가 되어 정렬이 깨짐
정렬 리스트는 반드시 당겨야 함 -> swap-with-last(O(1) 트릭)을 쓸 수 없음

실행 결과

```
[PASS] removeItem(3): (True, [1, 2, 4, 5])
[PASS] removeItem(10): (False, [1, 2, 3, 4, 5])
```

문제 4 · 소개

updateItem( ) — 값을 바꾸되 정렬을 지키기

- data[pos]를 새 값으로 그냥 덮어쓰면 순서가 깨질 수 있음
- 예: [1, 3, 5]에서 3을 9로 바꾸면 [1, 9, 5] -> 정렬이 아님

완성할 코드 (practice.py)

```
def updateItem(self, old_value, new_value):
    removed = NEED_TO_SOLVE
    if not removed:
        return False
    NEED_TO_SOLVE
    return True
```

- 이미 만든 연산 두 개만 순서대로 호출하면 됨 (직접 인덱스를 옮기지 않아도 됨)

⏱️ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

문제 4 · 풀이 · updateItem( )

지우고, 다시 정렬 위치에 넣기

풀이 코드

```
removed = self.removeItem(old_value)
if not removed:
    return False
self.insertItem(new_value)
return True
```

updateItem(3, 10) 예시

```
시작   [1, 2, 3, 4, 5]
제거   [1, 2, 4, 5]
       (3을 지움)
삽입   [1, 2, 4, 5, 10]
       (정렬 위치에 넣음)
```

```
removeItem은 O(N), insertItem도 O(N) -> updateItem은 O(N) + O(N) = O(N)
새로 만든 코드 없이, 이미 만든 연산을 조합한 것 -> 재사용(reuse)의 예시
```

실행 결과

```
[PASS] updateItem(3, 10): (True, [1, 2, 4, 5, 10])
[PASS] updateItem(99, 1): (False, [1, 2, 3, 4, 5])
```

CHECKPOINT

문제 1~4를 다 채우면 나오는 화면

python3 practice.py

```
=== 체크포인트 ===
  [PASS] insertItem 3,5,1,4,2 삽입: [1, 2, 3, 4, 5]
  [PASS] findItem(4): 3
  [PASS] findItem(10): -1
  [PASS] removeItem(3): (True, [1, 2, 4, 5])
  [PASS] removeItem(10): (False, [1, 2, 3, 4, 5])
  [PASS] updateItem(3, 10): (True, [1, 2, 4, 5, 10])
  [PASS] updateItem(99, 1): (False, [1, 2, 3, 4, 5])
결과: 전부 통과
```

- FAIL이 남아 있으면 실험은 실행되지 않음 -> 먼저 전부 PASS로 만들기
- 막히면 practice_answer.py와 비교해서 어느 줄이 다른지 찾기

문제 5 · 소개

정렬 vs 비정렬 비교 실험 — 무엇을 재는가

- 실험 1: 없는 값을 찾는다 -> SortedList(이진 탐색) vs UnsortedList(선형 탐색)
- 실험 2: 새 값 삽입 -> SortedList(정렬 유지) vs UnsortedList(그냥 append)
- N을 1000 -> 2000 -> 4000 -> 8000 -> 16000으로 2배씩 키우며 반복

실험 2의 삽입 방식 (이미 작성됨, 각자 따로 측정)

```
# 항상 지금까지 넣은 값보다 작은 값을 삽입 -> 위치가 맨 앞으로 확정됨
lst_s.insertItem(-k - 1)   # SortedList: 맨 앞부터 전부 한 칸씩 밀어야 함
lst_u.appendItem(-k - 1)   # UnsortedList: 그냥 뒤에 붙이면 끝
```

- 결과를 보기 전에 예상해보기 — findItem과 insertItem, 각각 어느 쪽이 유리할까?

문제 5 · 결과 1

실험 1 — findItem: 이진 탐색 vs 선형 탐색

측정 결과 (없는 값을 찾는 최악의 경우, 1회 평균, 실행 환경마다 값은 다름)

```
       N | SortedList |  UnsortedList
-------------------------------------
    1000 |     0.540us |        47.0us
    4000 |     0.694us |       197.9us
   16000 |     0.851us |       782.6us
```

```
SortedList : N이 16배가 되어도 0.5us -> 0.85us. 거의 그대로 -> O(log N)
UnsortedList : N이 16배가 되면 시간도 약 16배 -> 끝까지 훑어야 함 -> O(N)
```

- 측정값은 컴퓨터마다 다름. 절대 시간이 아니라 'N이 커질 때 몇 배가 되는가'를 보는 것

실험 2 — 새 값 삽입: 정렬 유지 vs 그냥 append

측정 결과 (1회 평균, 실행 환경마다 값은 다름)

```
       N | SortedList |  UnsortedList
-------------------------------------
    1000 |      68.5us |       0.177us
    4000 |     256.5us |       0.185us
   16000 |     982.7us |       0.176us
```

```
SortedList : N이 16배가 되면 시간도 약 14~16배 -> 정렬 유지를 위해 전부 밀어야 함 -> O(N)
UnsortedList : N이 커져도 0.17~0.18us 그대로 -> 순서 상관 없이 뒤에 붙임 -> O(1)
```

- 실험 1에서 이긴 SortedList가, 실험 2에서는 UnsortedList에 짐 -> 정렬의 대가

문제 5 · 해설

왜 이런 트레이드오프가 생기는가

- 정렬 리스트는 저장할 때부터 순서를 지킴 -> 탐색 때 절반씩 버릴 수 있음(O(log N))
- 비정렬 리스트는 순서를 안 지킴
- 탐색은 끝까지 훑어야 하지만(O(N)), 삽입은 아무 데나 붙이면 됨(O(1))

연산별 승자

연산	SortedList	UnsortedList	이기는 쪽
findItem(탐색)	**O(log N)**	**O(N)**	**SortedList**
insertItem(삽입)	**O(N)**	**O(1)**	**UnsortedList**

```
탐색이 잦은 데이터는 정렬 리스트, 삽입/삭제가 잦은 데이터는 비정렬 리스트가 유리함
'정렬 상태를 유지하는 비용'과 '탐색을 빠르게 하는 이득'은 항상 맞바꾸는 관계
```

SUMMARY

오늘 구현한 연산의 Big-O

연산	하는 일	SortedList	UnsortedList(4주차)
`insertItem`	값을 저장 위치에 맞게 삽입	`O(N)`	`O(1) (improved)`
`findItem`	값의 위치를 찾음	`O(log N)`	`O(N)`
`removeItem`	찾아서 지움	`O(N)`	`O(N)`
`updateItem`	값을 바꾸되 정렬 유지	`O(N)`	-
`getItem / size / clear`	위치를 알고 접근하거나 length만 조작	`O(1)`	`O(1)`

정렬을 유지하면 탐색이 O(log N)으로 빨라지지만, 삽입은 O(N)으로 느려짐

무엇을 자주 하느냐(탐색 vs 삽입)에 따라 자료구조 선택이 달라짐

5주차 실습 끝

다음 시간 — 정렬 알고리즘